WEEK 3 PRACTICAL

MongoDB Connectivity and LLM Monitoring

1. Aim

To build a Python application that connects to Large Language Model (LLM) providers such as Gemini and Ollama, measures request details, calculates estimated API cost where applicable, and stores call logs in MongoDB.

2. Software and Tools Required

- macOS
- Visual Studio Code
- Python 3
- MongoDB Community Server
- MongoDB Compass
- Ollama with the llama3.2 model
- Gemini API key (for testing the Gemini provider)

3. Project Folder Structure

The project was created in VS Code with the following structure:

Project structure

```
WEEK-3/
├── .env
├── cost_calculator.py
├── llm_client.py
├── main.py
├── mongo_logger.py
└── requirements.txt
```

The .venv folder contains the local Python virtual environment. It should not be uploaded to GitHub. The .env file contains the API key and should also remain private.

4. Procedure

1. Create and open the project: Create a folder named WEEK-3 and open it in Visual Studio Code.

2. Create a virtual environment: Open the VS Code terminal and run: python3 -m venv .venv

3. Activate the environment: Run: source .venv/bin/activate

4. Upgrade pip: Run: python -m pip install --upgrade pip

5. Install dependencies: Create requirements.txt and run: pip install -r requirements.txt

6. Install and start MongoDB: Install MongoDB Community Server and start the local MongoDB service.

7. Open MongoDB Compass: Connect to mongodb://localhost:27017/. Create or use the database llm_monitoring and collection call_logs.

8. Install and start Ollama: Install Ollama, download the llama3.2 model using ollama pull llama3.2, and keep Ollama running.

9. Add the Gemini API key: Create a .env file and set GEMINI_API_KEY to your own key. Never commit this file to GitHub.

10. Run the application: In the activated virtual environment, run: python main.py. Select provider 1 for Gemini or 2 for Ollama, then enter a prompt.

11. Verify MongoDB logs: Open llm_monitoring → call_logs in MongoDB Compass and verify that request records are stored.

5. Source Code

The following code listing documents the project files. The Gemini model identifier and pricing shown in the code are configurable; verify the model name and applicable rates against the Google AI account/documentation used for the actual run.

5.1 cost_calculator.py

```
# cost_calculator.py

PRICING = {
    "gemini-3.8-flash": {
        "input": 0.75,
        "output": 3.75
    }
}

def calculate_cost(provider, model, input_tokens, output_tokens):
    # Local Ollama inference does not have a per-token API charge.
    if provider == "OllamaProvider":
        return 0.0

    rates = PRICING.get(model)
    if rates is None:
        return None

    input_cost = input_tokens * rates["input"] / 1_000_000
    output_cost = output_tokens * rates["output"] / 1_000_000

    return round(input_cost + output_cost, 10)
```

5.2 llm_client.py

```
# llm_client.py

from abc import ABC, abstractmethod
from google import genai
from google.genai import types
import requests
import time
from cost_calculator import calculate_cost
```

```
class LLMProvider(ABC):
    @abstractmethod
    def generate(self, prompt):
        pass


class GeminiProvider(LLMProvider):
    def __init__(self, api_key):
        self.client = genai.Client(
            api_key=api_key,
            http_options=types.HttpOptions(timeout=120000)
        )
        self.model = "gemini-3.8-flash"

    def generate(self, prompt):
        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt
        )
        usage = response.usage_metadata
        input_tokens = usage.prompt_token_count if usage else 0
        output_tokens = usage.candidates_token_count if usage else 0

        return {
            "response": response.text,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens
        }


class OllamaProvider(LLMProvider):
    def __init__(self, model="llama3.2"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
```

```
        response = requests.post(
            self.url,
            json={"model": self.model, "prompt": prompt, "stream": False},
            timeout=120
        )
        response.raise_for_status()
        data = response.json()

        return {
            "response": data["response"],
            "input_tokens": data.get("prompt_eval_count", 0),
            "output_tokens": data.get("eval_count", 0)
        }


class LLMClient:
    def __init__(self, provider, logger):
        self.provider = provider
        self.logger = logger

    def generate(self, prompt):
        max_attempts = 3
        start_time = time.perf_counter()
        last_error = None

        for attempt in range(1, max_attempts + 1):
            try:
                result = self.provider.generate(prompt)
                latency = round((time.perf_counter() - start_time) * 1000, 2)
                input_tokens = result["input_tokens"]
                output_tokens = result["output_tokens"]
                model = self.provider.model
                provider_name = type(self.provider).__name__

                cost = calculate_cost(
                    provider_name, model, input_tokens, output_tokens
                )
```

```
            log_data = {
                "provider": provider_name,
                "model": model,
                "prompt": prompt,
                "input_tokens": input_tokens,
                "output_tokens": output_tokens,
                "total_tokens": input_tokens + output_tokens,
                "latency_ms": latency,
                "cost": cost,
                "attempts": attempt,
                "status": "success",
                "error": None
            }

            try:
                self.logger.log_call(log_data)
            except Exception as log_error:
                print("MongoDB logging error:", log_error)

            return result["response"]

        except Exception as error:
            last_error = error
            status_code = getattr(error, "status_code", None)

            retryable = (
                isinstance(error, (
                    requests.exceptions.Timeout,
                    requests.exceptions.ConnectionError
                ))
                or status_code == 429
                or (isinstance(status_code, int) and status_code >= 500)
                or (
                    isinstance(error, requests.exceptions.HTTPError)
                    and error.response is not None
                    and (
                        error.response.status_code == 429
                        or error.response.status_code >= 500
```

```
                )
            )
        )

        if not retryable or attempt == max_attempts:
            break

        wait_time = 2 ** (attempt - 1)
        print(f"Attempt {attempt} failed. Retrying in {wait_time} seconds...")
        time.sleep(wait_time)

    latency = round((time.perf_counter() - start_time) * 1000, 2)
    failed_log = {
        "provider": type(self.provider).__name__,
        "model": self.provider.model,
        "prompt": prompt,
        "latency_ms": latency,
        "attempts": attempt,
        "status": "failed",
        "error": str(last_error)
    }

    try:
        self.logger.log_call(failed_log)
    except Exception as log_error:
        print("MongoDB logging error:", log_error)

    raise last_error
```

5.3 main.py

```
# main.py

import os
from dotenv import load_dotenv
from llm_client import GeminiProvider, OllamaProvider, LLMClient
from mongo_logger import MongoLogger

load_dotenv(override=True)
```

```
print("==================================")
print("     WEEK 3: LLM MONITORING")
print("==================================")
print("1. Gemini")
print("2. Ollama")

choice = input("\\nSelect provider (1/2): ").strip()
logger = None

try:
    logger = MongoLogger()

    if choice == "1":
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError("Gemini API key not found in .env")
        provider = GeminiProvider(api_key)

    elif choice == "2":
        provider = OllamaProvider("llama3.2")

    else:
        raise ValueError("Invalid selection. Choose 1 or 2.")

    client = LLMClient(provider, logger)
    prompt = input("\\nEnter your question: ")
    answer = client.generate(prompt)

    print("\\n========== AI RESPONSE ==========")
    print(answer)
    print("\\n========== COMPLETED ==========")

except Exception as error:
    print("\\nApplication error:", error)

finally:
    if logger is not None:
```

```
        logger.close()
```

5.4 mongo_logger.py

```
# mongo_logger.py

from pymongo import MongoClient
from datetime import datetime, timezone



class MongoLogger:
    def __init__(self):
        self.client = MongoClient("mongodb://localhost:27017/")
        self.db = self.client["llm_monitoring"]
        self.collection = self.db["call_logs"]

    def log_call(self, log_data):
        log_data["timestamp"] = datetime.now(timezone.utc)
        self.collection.insert_one(log_data)
        print("Call details saved in MongoDB!")

    def close(self):
        self.client.close()
```

5.5 requirements.txt

```
pytest
google-genai
python-dotenv
pymongo
requests
```

5.6 .env (example only)

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY_HERE
```

Security note: Replace the placeholder with the actual API key only in the local .env file. Do not publish a real API key or the .env file to GitHub.

6. Screenshots and Observations

The following screenshots were captured during setup and execution on macOS.

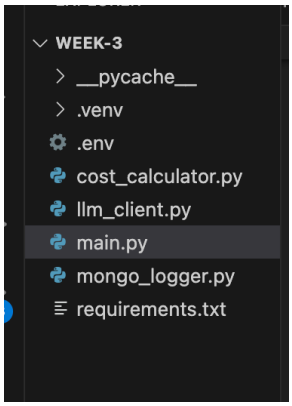


Figure 1. VS Code project folder showing the Python source files and requirements.txt.

```
sahasra@Sah week-3 % python3 -m venv .venv
sahasra@Sah week-3 % source .venv/bin/activate
(.venv) sahasra@Sah week-3 % python -m pip install --upgrade pip
Requirement already satisfied: pip in ./.venv/lib/python3.14/site-packages (25.2)
Collecting pip
  Downloading pip-26.3-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.3-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 2.0 MB/s  0:00:01
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-26.3
(.venv) sahasra@Sah week-3 % pip install -r requirements.txt
Collecting pytest (from -r requirements.txt (line 2))
  Using cached pytest-9.2.0-py3-none-any.whl.metadata (7.9 kB)
Collecting google-genai (from -r requirements.txt (line 3))
  Downloading google_genai-3.2.0-py3-none-any.whl.metadata (60 kB)
Collecting python-dotenv (from -r requirements.txt (line 4))
  Using cached python_dotenv-1.2.4-py3-none-any.whl.metadata (29 kB)
Collecting pymongo (from -r requirements.txt (line 5))
```

Figure 2. macOS terminal showing creation and activation of the virtual environment and dependency installation.

```
(.venv) sahasra@Sah week-3 % python main.py
================================
     WEEK 3: LLM MONITORING
================================
1. Gemini
2. Ollama

Select provider (1/2): 2

Enter your question: what is ai?
Call details saved in MongoDB!

========== AI RESPONSE ==========
AI, or Artificial Intelligence, refers to the development of computer systems that can perform tasks that
typically require human intelligence, such as:

1. Learning: AI systems can learn from data and improve their performance over time.
2. Problem-solving: AI systems can analyze problems and provide solutions.
3. Reasoning: AI systems can draw conclusions based on data and rules.
4. Perception: AI systems can interpret and understand data from sensors and other sources.
```

Figure 3. Successful Ollama execution: the application generated an AI response and saved call details in MongoDB.

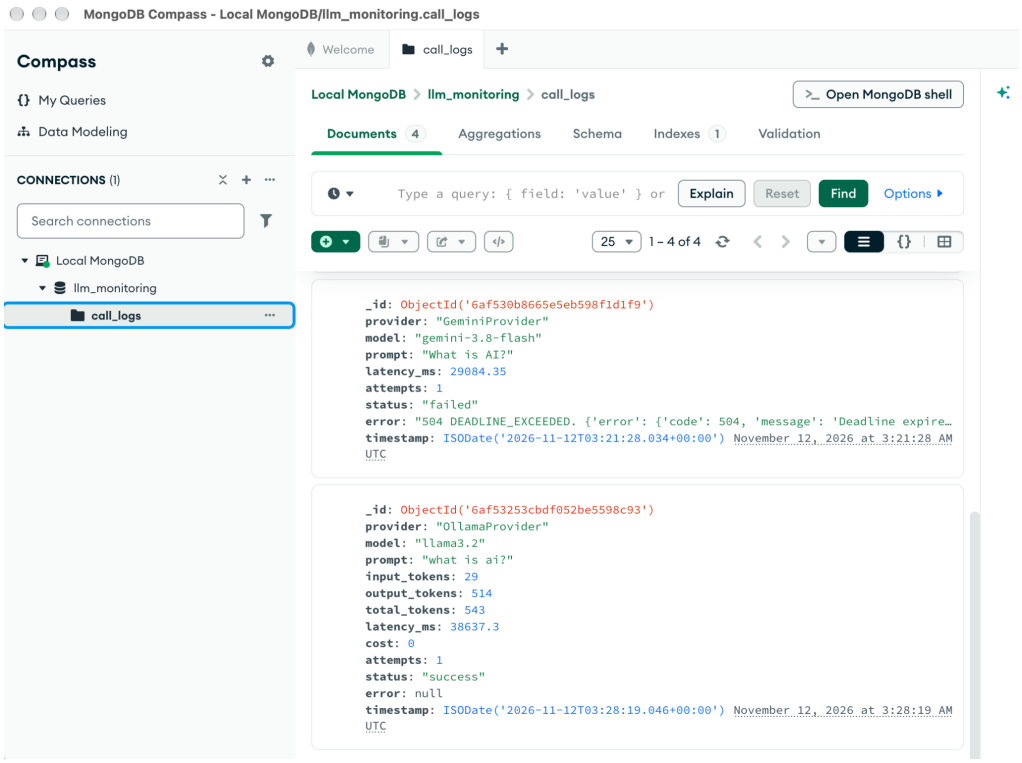


Figure 4. MongoDB Compass showing the llm_monitoring database and call_logs collection, including successful Ollama and failed Gemini records.

7. Execution Results

Ollama: The application was run with provider option 2 and the prompt “what is ai?”. The llama3.2 model returned a response, and the terminal displayed “Call details saved in MongoDB!”. MongoDB Compass also shows a successful OllamaProvider record with token counts, latency, cost 0, and status success.

Gemini: A GeminiProvider record is visible in MongoDB Compass with status failed and a 504 DEADLINE_EXCEEDED error. This indicates that the Gemini request did not complete within the deadline during this test. The failure was recorded in MongoDB, demonstrating that failed calls are also logged. The Gemini run should be repeated after checking the API model identifier, API access, and network availability.

9. Conclusion

The Python application was configured to communicate with LLM providers and record request information in MongoDB. The Ollama llama3.2 test completed successfully and its execution details were stored in the call_logs collection. A Gemini timeout was also recorded as a failed call, showing that the monitoring system captures both successful and failed requests.